

PHP vs JavaScript

Cecha / Operacja	PHP (Obiektowy)	JavaScript (ES6+ Obiektowy)
Miejsce uruchamiania	Serwer (XAMPP / Apache)	Przeglądarka (Klient)
Definiowanie klasy	class Nazwa { ... }	class Nazwa { ... }
Konstruktor	public function __construct()	constructor()
Tworzenie obiektu	\$obj = new Nazwa();	const obj = new Nazwa();
Dostęp do składowych	Operator -> (np. \$this->nazwa)	Kropka . (np. this.nazwa)
Modyfikatory dostępu	public, protected, private	Brak słów kluczowych (prywatne z #)
Dziedziczenie	class A extends B	class A extends B
Wywołanie konstruktora klasy bazowej	parent::__construct()	super()

Szybka ściągawka składniowa na egzamin

W zadaniach egzaminacyjnych INF.03 najczęściej musisz napisać prostą klasę reprezentującą np. produkt, użytkownika lub auto, a następnie wyświetlić jej dane.

W **PHP** przy tworzeniu właściwości w klasach możemy określić ich typ danych (**int**, **string** czy **float**), np. **public int \$wiek**; Natomiast jest to nieobowiązkowe, a przypisanie błędnej wartości spowoduje przerwanie działania skryptu błędem

1. Definiowanie Klasy i Konstruktora

PHP: Wymaga znaku \$ przed właściwościami. Pamiętaj o podwójnym podkreśleniu w konstruktorze.

```
class Produkt {  
    public $nazwa;  
    public $cena;  
  
    public function __construct($nazwa, $cena) {  
        $this->nazwa = $nazwa;  
        $this->cena = $cena;  
    }  
}
```

JavaScript: Nie deklarujesz zmiennych przed konstruktorem (chyba, że używasz pól prywatnych).

```
class Produkt {  
    constructor(nazwa, cena) {  
        this.nazwa = nazwa;  
        this.cena = cena;  
    }  
}
```

2. Tworzenie Obiektu i Metody

PHP: Używasz operatora strzałki ->. Nie pisz \$ po strzałce (częsty błąd: \$this->\$nazwa zamiast \$this->nazwa).

```
class Produkt {
    public $nazwa;
    public $cena;

    public function __construct($nazwa, $cena) {
        $this->nazwa = $nazwa;
        $this->cena = $cena;
    }
    public function wyswietl() {
        return "Produkt: " . $this->nazwa;
    }
}
$p = new Produkt("Książka", 45);
```

JavaScript: Używasz standardowej notacji z kropką.

```
class Produkt {
    // ... (konstruktor jak wyżej)
    wyswietl() {
        return `Produkt: ${this.nazwa}`;
    }
}
const p = new Produkt("Książka", 45);
console.log(p.wyswietl());
```

3. Dziedziczenie (Wymagane na 100% poprawności)

PHP:

```
class Ksiazka extends Produkt {  
    public $autor;  
    public function __construct($nazwa, $cena, $autor) {  
        parent::__construct($nazwa, $cena);  
        $this->autor = $autor;  
    }  
}
```

JavaScript: Pamiętaj, że `super()` musi być wywołane jako pierwsze w konstruktorze.

```
class Ksiazka extends Produkt {  
    constructor(nazwa, cena, autor) {  
        super(nazwa, cena);  
        this.autor = autor;  
    }  
}
```

Pułapki egzaminacyjne, na które trzeba uważać

- **W PHP** zapomnienie o `parent::__construct()` w klasie pochodnej zresetuje lub pominie przypisywanie bazowych właściwości.
- **W JS** próba użycia `this.nazwa` przed wywołaniem `super()` w konstruktorze klasy potomnej wyrzuci błąd `ReferenceError`.
- **W PHP** modyfikatory `public/private` są pisane wprost. W nowoczesnym JS pole prywatne tworzy się stawiając płatek przed nazwą (np. `#cena`), ale na egzaminie INF.03 rzadko jest to wymagane – najczęściej CKE operuje na polach publicznych.
- **Łączenie ciągów:** W PHP używasz kropki (`.`), a w JavaScript plusa (`+`) lub szablonów literowych (backtick ```).

Łączenie ciągów

PHP

```
class Osoba{
    public $imie;
    public $nazwisko;

    public function przedstawSie() {
        return "Nazywam się ".$this->imie." ".$this->nazwisko.".";
    }
}
```

JavaScript

```
class Osoba{

    przedstawSiePlus() {
        return "Nazywam się " + this.imie + " " + this.nazwisko + ".";
    }
    przedstawSieSzablon() {
        return `Nazywam się ${this.imie} ${this.nazwisko}.`;
    }
}
```